

Kimi K3 三项核心技术

聚焦 KDA、Attention Residuals、Stable MoE：分别从序列长度、模型深度、专家宽度三个维度解释 K3 的结构跃迁。

KDA / 1M Context

Block AttnRes

Stable LatentMoE

Quantile Balancing

0. 结论速览

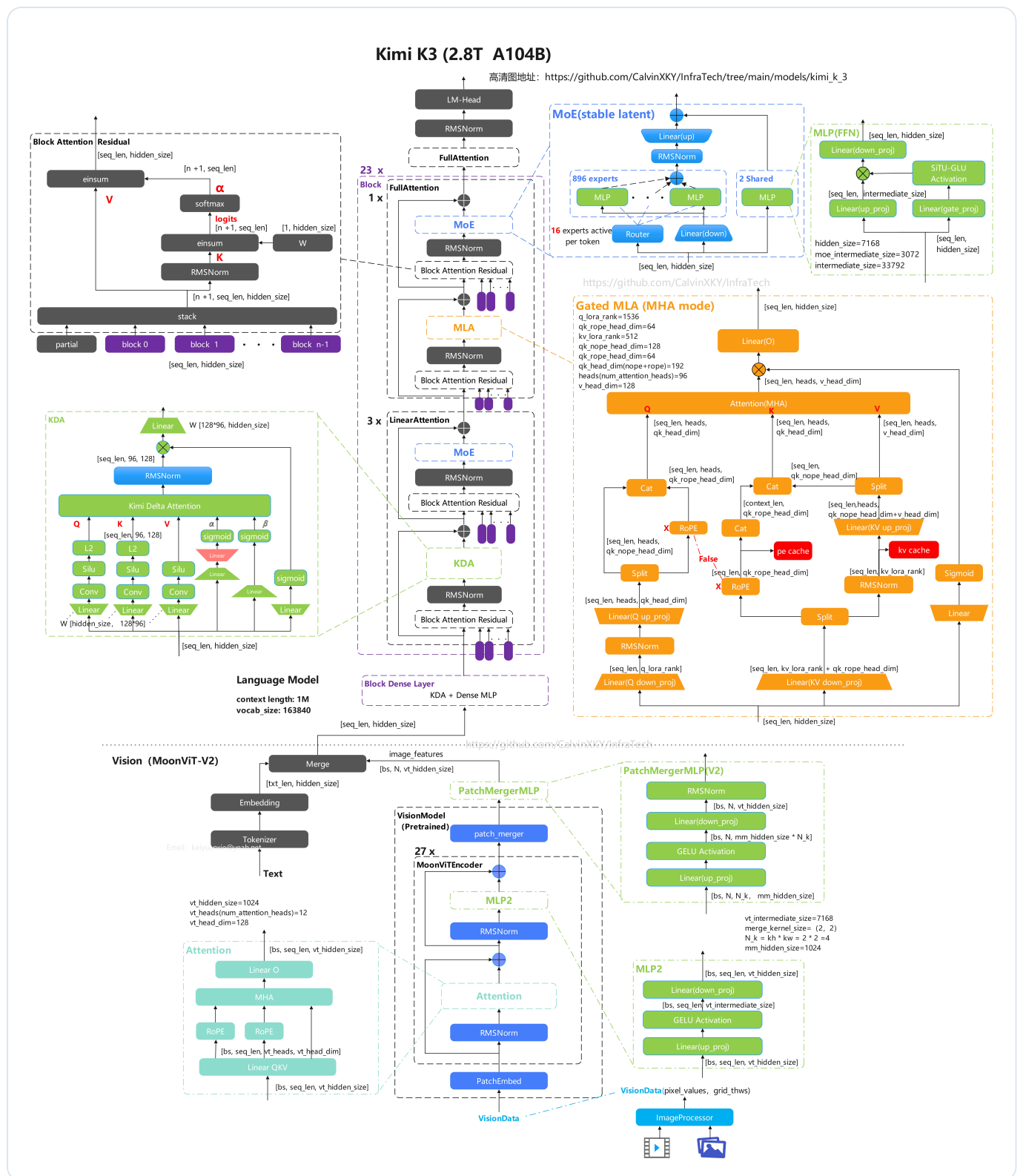
0.1 三项技术在 K3 中分别解决什么

技术	对应维度	K3 中的位置	主要解决的问题	关键创新
Kimi Delta Attention (KDA)	序列维度	69 个 KDA layers，与 24 个 Gated MLA layers 组成 3:1 hybrid attention	1M context 下 full attention 的 KV cache 和 prefill/decode 成本过高	用固定大小 recurrent state 替代随序列增长的 KV cache，并用 Gated MLA 周期性补全全局交互
Attention Residuals (AttnRes)	深度维度	K3 93 层 backbone 中的 Block AttnRes	深层模型中早期表示被层层 residual 压缩，跨层信息检索能力弱	把 “attention over tokens” 的思想搬到 “attention over layers”，让每层选择性检索前序层表示
Stable LatentMoE / Stable MoE	宽度与专家维度	每个 attention layer 后接 Stable LatentMoE feed-forward network	896 routed experts、Top-16 激活时，普通 MoE 容易出现激活爆炸、专家负载不均和通信开销上升	Latent routed path + RMSNorm + SiTU-GLU + Quantile Balancing + MoonEP，使极大专家池稳定训练和服务

0.2 K3 不是 K2.x 后训练增强，而是结构跳变

项	Kimi K2	Kimi K3	变化含义
总参数	1.04T	2.78T	容量显著扩大
激活参数	32.6B	104.2B	单 token 计算也明显上升
层数	61	93	深度增加，需要更强跨层信息流

项	Kimi K2	Kimi K3	变化含义
context	128K	1M	序列维度进入百万 token
attention	61 MLA	69 KDA + 24 MLA	从纯 MLA 转向 hybrid KDA-MLA
routed experts	384	896	专家空间扩大 133%
active experts/token	8	16	每 token 专家混合能力增强
shared experts	1	2	保留更强通用 full-width path
activation	SwiGLU	SiTU-GLU	为低精度和极大 MoE 稳定性做约束



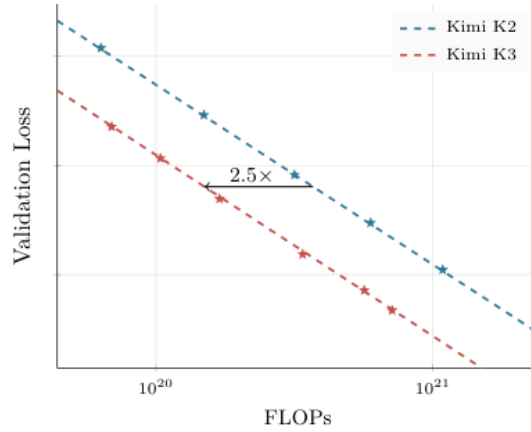


Figure 7: Fitted scaling-law curves for Kimi K2 and Kimi K3. Kimi K3 achieves $2.5\times$ gain in scaling efficiency over Kimi K2.

Table 1: Architectural comparison between Kimi K2 and Kimi K3.

	Kimi K2	Kimi K3	Δ
Architecture	MoE	MoE	–
#Layers	61	93	$\uparrow 52\%$
Total Parameters	1.04T	2.78T	$\uparrow 167\%$
Activated Parameters	32.6B	104.2B	$\uparrow 220\%$
Hidden Dimension	7,168	7,168	=
Latent MoE Dimension	–	3584 (0.5 $\times$)	–
MoE Hidden Dimension per Expert	2,048	3,072	$\uparrow 50\%$
Routed Experts	384	896	$\uparrow 133\%$
Experts Active per Token	8	16	$\uparrow 100\%$
Shared Experts	1	2	$\uparrow 100\%$
Attention Heads	64	96	$\uparrow 50\%$
Number of Dense Layers	1	1	=
Vocabulary Size	160K	160K	=
Training Context Length	128K	1M	$8\times$
Attention Mechanism	MLA	Hybrid KDA–MLA	–
Activation Function	SwiGLU	SiTU–GLU	–
Attention-Layer Composition	61 MLA	69 KDA + 24 MLA	–
Number of MTP Layers	1 layer	1 layer	=
Total Parameters of ViT	–	401M	–
#ViT Layers	–	27 layers	–
Patch Size of ViT	–	14	–
#Attention Heads of ViT	–	12	–

3.3 Training Recipe

Kimi K3 adopts a native multimodal training strategy in which language and vision are jointly optimized from the start of training, rather than grafting a vision encoder onto a pre-trained language model through a post-hoc alignment stage. Under this paradigm, visual and textual tokens are interleaved within a single next-token prediction objective, enabling the shared backbone to learn unified multimodal representations from the outset.

We optimize the model using the Per-Head Muon optimizer (§ 2.5) together with the weight-clipping mechanism introduced in Kimi K2, while adopting QB (§ 2.3.3) for MoE load balancing. We use a cosine learning rate schedule with a 1% linear warmup. Weight decay is set to 0.1 throughout.

图：Kimi K3 technical report 第 11 页。Figure 7 和 Table 1 给出 K2 到 K3 的 scaling efficiency 与结构差异。

1. KDA：把 1M context 从 KV cache 问题改造成 recurrent state 问题

1.1 K3 为什么需要 KDA

K3 的目标上下文是 1M tokens。如果继续使用标准 full attention 或只靠 MLA，成本压力主要来自两块：

压力	说明
KV cache 随长度线性增长	每层都需要保存历史 token 的 key/value 或 latent KV；1M context 下 cache 成本直接进入服务瓶颈
prefill 计算和 prefix reuse 难度上升	长输入一旦 cache miss，prefill 代价极高；如果 prefix cache 颗粒度过粗，短请求和部分匹配都无法复用

KDA 的核心转向是：不再为每个历史 token 保存一份 KV，而是用固定大小的 recurrent state 承载历史信息。代价是状态更新存在串行依赖，需要专门 kernel 和 context parallelism；收益是 cache 体积、跨节点传输和 prefix 复用都可以重新设计。

1.2 KDA 的基本 recurrence

K3 technical report 对单头 KDA 的定义如下。设 hidden state 为 $x_t \in \mathbb{R}^d$ ，query/key/value 分别为 $q_t, k_t \in \mathbb{R}^{d_k}$ 、 $v_t \in \mathbb{R}^{d_v}$ ，recurrent state 为 $S_t \in \mathbb{R}^{d_k \times d_v}$ 。

$$S_t = (I - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^\top$$
$$\tilde{o}_t = S_t^\top q_t$$

其中：

符号	含义
$\alpha_t \in (0, 1)^{d_k}$	channel-wise one-step retention factor，决定每个 key channel 的遗忘/保留强度
$\beta_t \in (0, 1)$	delta-rule write strength，决定当前 token 写入 recurrent state 的强度
$I - \beta_t k_t k_t^\top$	delta-rule correction，用当前 key 对旧状态做校正
$\beta_t k_t v_t^\top$	当前 token 写入项

这个公式的直觉是：KDA 不只是累加历史 value，而是带有“遗忘、校正、写入”的状态机。它比普通线性注意力更像可学习的有限状态 memory。

1.3 K3 相比 Kimi Linear 的关键改动

Kimi Linear 已经验证 KDA + MLA 的 3:1 hybrid 路线。K3 继承这条路线，但做了两个重要改变。

改动	Kimi Linear	Kimi K3	作用
decay parameterization	negative-Softplus, log-decay 下界为 $-\infty$	scaled sigmoid，把 log-decay 限制在 $(g_{\min}, 0)$	避免 chunk 内 reciprocal cumulative decay 过大，减少 BF16 溢出和 diagonal tile 特殊路径
output gate	low-rank gate	full-rank input-dependent gate	提高通道级输出调制能力，与 Gated MLA 的 full-rank gate 对齐

K3 的 lower-bounded decay 写法是：

$$g_t^h = g_{\min} \operatorname{Sigmoid}\left(e^{A^h} z_t^h\right) \in (g_{\min}, 0)^{d_k}$$
$$\alpha_t^h = \exp(g_t^h) \in (e^{g_{\min}}, 1)^{d_k}$$

K3 固定 $g_{\min} = -5$ 。因此每个 retention factor 都满足：

$$\alpha_{t,j}^h > e^{-5} \approx 6.7 \times 10^{-3}$$

对 16-token tile，累计 log-decay 落在 $(-80, 0)$ ，reciprocal rescaling 小于 e^{80} ，仍在 BF16 动态范围内。这样 diagonal 和 off-diagonal causal tiles 都可以走 dense Tensor Core GEMM，减少原先 position-pair diagonal path 的瓶颈。

K3 的 full-rank output gate 为：

$$y_t = W_o [\operatorname{Sigmoid}(W_g x_t) \odot \operatorname{RMSNorm}(\tilde{o}_t)]$$

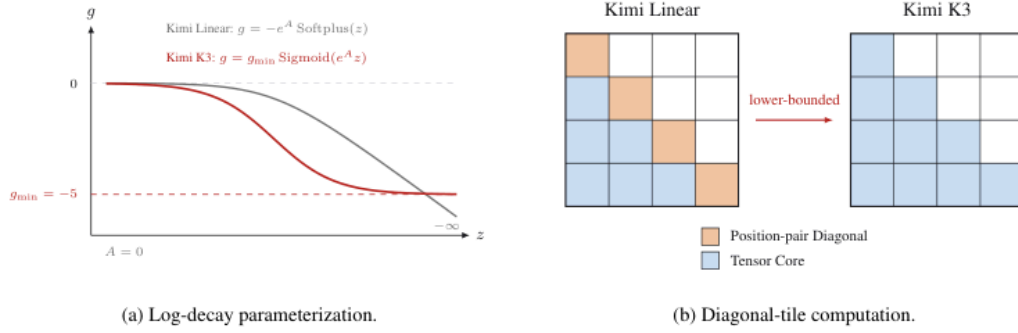


Figure 3: Lower-bounded decay and its effect on chunkwise KDA computation. (a) Kimi Linear uses an unbounded negative-Softplus mapping, whereas Kimi K3 bounds the log-decay with a scaled sigmoid; the curves show $A = 0$ and $g_{\min} = -5$. (b) Kimi Linear evaluates each diagonal tile with an explicit position-pair computation, while the bounded range in Kimi K3 allows all causal tiles to use dense Tensor Core matrix multiplications.

Kimi K3 addresses this bottleneck by changing the mapping from the decay logits z_t^h to the per-step log-decay g_t^h . Following GDN and Mamba-2, Kimi Linear uses the negative-Softplus mapping $g_t^h = -e^{A_h} \text{Softplus}(z_t^h) \in (-\infty, 0)^{d_k}$ [139, 24, 64]. Kimi K3 instead uses a scaled sigmoid to bound the log-decay from below:

$$\begin{aligned} g_t^h &= g_{\min} \text{Sigmoid}(e^{A_h} z_t^h) \in (g_{\min}, 0)^{d_k}, \\ \alpha_t^h &= \exp(g_t^h) \in (e^{g_{\min}}, 1)^{d_k}, \end{aligned} \quad (5)$$

where A_h is a learnable per-head log-scale and $g_{\min} = -5$ is fixed. We initialize $A_h = 0$, and each bias b_α^h is initialized following [64, 24, 139]. With $g_{\min} = -5$, every retention factor satisfies $\alpha_{t,j}^h > e^{-5} \approx 6.7 \times 10^{-3}$, and the cumulative log-decay over a 16-token tile lies in $(-80, 0)$. The corresponding reciprocal rescaling factor is therefore smaller than e^{80} and remains within the BF16 dynamic range. This finite range allows both diagonal and off-diagonal tiles to use dense Tensor Core matrix multiplications, eliminating the position-pair diagonal path. This parameterization is closely related to the lower-bounded recurrence gates in prior work [98, 27, 92]. Fig. 3 illustrates the change in decay parameterization and its computational consequence.

Full-rank gate Finally, Kimi K3 changes KDA’s output gate from the low-rank parameterization used by Kimi Linear [64] to an input-dependent full-rank projection. After applying head-wise RMSNorm [147] to the recurrent output, KDA applies data-dependent output gating [100]:

$$y_t = \mathbf{W}_o [\text{Sigmoid}(\mathbf{W}_g x_t) \odot \text{RMSNorm}(\tilde{o}_t)]. \quad (6)$$

2.1.2 Gated MLA

Multi-head Latent Attention (MLA), introduced in DeepSeek-V2 [28], compresses the key–value representation of each token into a low-dimensional latent vector $c_t = \mathbf{W}_c x_t$. Instead of caching full head-specific keys and values, MLA caches c_t and reconstructs the content keys and values through learned up-projections during attention computation. This factorization reduces the KV-cache footprint while retaining global token-to-token attention. MLA was subsequently adopted by Kimi K2 and Kimi K2.5 [59, 60], and Kimi K3 retains it in the periodic global-attention layers.

Unlike Kimi K2 and Kimi K2.5, Kimi K3 follows the hybrid design of Kimi Linear [64] and applies No Position Encoding (NoPE) to all MLA layers. Consequently, no explicit positional encoding is applied to their queries or keys. The intervening KDA layers provide position-sensitive and recency-aware sequence mixing, while the MLA layers provide unrestricted global content interaction. This separation also avoids modifying positional-encoding parameters when extending the context length, such as retuning a RoPE frequency base or applying YaRN [93].

In addition, Kimi K3 augments MLA with an input-dependent, channel-wise full-rank output gate. Let $\tilde{o}_t$ denote the ungated MLA output at position t ; the gated output is

$$y_t = \mathbf{W}_o [\text{Sigmoid}(\mathbf{W}_g x_t) \odot \tilde{o}_t]. \quad (7)$$

图：Kimi K3 technical report 第 5 页，包含 KDA lower-bounded decay、Gated MLA 和 AttnRes 相邻定义。

1.4 为什么 K3 不是纯 KDA，而是 KDA + Gated MLA

K3 每个 block 中使用 3 个 KDA layers + 1 个 Gated MLA layer，整个 backbone 为 69 KDA + 24 Gated MLA，末尾额外放一个 Gated MLA，确保最后一层仍能执行全局 attention。

这个组合的取舍是：

纯 KDA 的优势	纯 KDA 的问题
cache 固定、长上下文成本低、递推状态适合 1M context	对精确 token-token 检索、needle 类任务和跨远距离证据绑定可能弱于 full/global attention

Gated MLA 的作用	说明
周期性提供 unrestricted global content interaction	补偿有限状态 memory 对精确全局交互的不足
使用 NoPE	K3 把位置/recency 信息交给 KDA，不再为 MLA queries/keys 加显式 positional encoding
full-rank gate	让每个 token 自适应调制从 global attention 读出的通道

Gated MLA 的输出门控为：

$$y_t = W_o [\text{Sigmoid}(W_g x_t) \odot \tilde{o}_t]$$

1.5 KDA 的系统共设计：FlashKDA、KCP、prefix cache

KDA 不是只改模型结构。K3 report 专门给了 KDA 的系统共设计。

系统点	解决的问题	K3 做法
FlashKDA	chunkwise KDA 训练/prefill 中，chunk 内并行与 chunk 间状态传播交替导致 SM 空闲	CUTLASS-based chunkwise kernel，重叠 intra-chunk computation 与 cross-chunk state propagation
intra-device context parallelism	超长 prefill 下，纯 TP 只分 heads，不能缩短 recurrent path	在单 rank 的 SM 内切分 sequence segment，并合成 exact initial state
KDA Context Parallelism (KCP)	跨设备 context parallelism 不能像普通 linear attention 那样直接求和状态	每段拆成 local zero-state contribution 和 cumulative transition，用 prefix scan 组合

系统点	解决的问题	K3 做法
KDA-aware prefix cache	KDA recurrent state 和 MLA KV cache 生命周期、大小、颗粒度都不同	统一 paged pool, MLA 细粒度 hash blocks, KDA 在 hash endpoint 保存 sparse checkpoints

KCP 的关键原因是 KDA 状态更新不是加法递推。写作：

$$S_t = M_t S_{t-1} + \beta_t k_t v_t^\top$$

其中：

$$M_t = (I - \beta_t k_t k_t^\top) \text{Diag}(\alpha_t)$$

所以某段序列对输入 state 的作用依赖 M_t 的连乘，不能只把“从零开始得到的 state”相加。K3 的做法是让每个 rank 计算两个本地片段：

片段	作用
$\tilde{S}$	本段从 $S = 0$ 开始生成的状态
$M^{T \leftarrow 1}$	本段对输入 state 的 cumulative transition

然后跨 rank all-gather 这些固定大小张量，用 associative prefix scan 重构每个 rank 的 incoming state。

independently of the incoming state and composed exactly afterward. An automatic SM-level context-parallel (CP) planner [143, 140] therefore partitions the sequence across the SMs of a single rank, evaluates the segment transitions in parallel, and merges them to recover each segment's exact initial state. In contrast to the cross-device KCP of §5.1.2, this parallelism is entirely intra-device and incurs no cross-device communication.

KDA decoding presents challenges distinct from those encountered during training and prefill. We discuss these challenges in detail in §5.4.2.

5.1.2 KDA Context Parallelism

The communication overhead of context parallelism differs fundamentally between softmax and linear attention. Softmax attention requires ranks to exchange key-value blocks whose size grows with the sequence length [73]. Linear attention instead carries the preceding context in a fixed-size recurrent state $\mathbf{S} \in \mathbb{R}^{d_k \times d_v}$. Prior context-parallel methods exploit the additive recurrence of vanilla linear attention by computing, on each rank, the state that the local tokens generate from $\mathbf{S} = \mathbf{0}$ and summing these local states over the preceding ranks to recover the incoming state [115, 114].

This direct summation, however, is insufficient for KDA. Recall from Eq. 1 that KDA updates its state as $\mathbf{S}_t = \mathbf{M}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$, where $\mathbf{M}_t := (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\alpha_t)$. KDA's delta rule applies the token-dependent matrix $\mathbf{M}_t$ to the incoming state before adding the current write. Consequently, the effect of a local sequence segment depends on the state entering that segment and cannot be determined from the state computed with $\mathbf{S} = \mathbf{0}$ alone.

To preserve this dependence, we introduce KDA Context Parallelism (KCP), which decomposes the effect of each segment into two locally computable quantities, a cumulative transition acting on the incoming state and a state generated locally from zero. Following the chunkwise notation of §2.1.1, we write $\mathbf{S}_{[i]}^t$ for the recurrent state within the segment of rank i after t local tokens, so that $\mathbf{S}_{[i]}^{T_i}$ denotes the state leaving rank i and entering rank $i+1$. We write $\tilde{\mathbf{S}}_{[i]}^t$ for the state of the same recurrence started instead from $\mathbf{S} = \mathbf{0}$. For an arbitrary state entering the $(i+1)$ -th of P context-parallel ranks, the state after t local tokens is

$$\begin{aligned} \mathbf{M}_{[i+1]}^{t \leftarrow 1} &:= \prod_{r \leftarrow 1}^t \mathbf{M}_r \in \mathbb{R}^{d_k \times d_k}, & \mathbf{S}_{[i+1]}^t &= \tilde{\mathbf{S}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \mathbf{S}_{[i]}^{T_i} \\ & & &= \tilde{\mathbf{S}}_{[i+1]}^t + \mathbf{M}_{[i+1]}^{t \leftarrow 1} \sum_{j=1}^i \left(\prod_{l \leftarrow j+1}^i \mathbf{M}_{[l]}^{T_l \leftarrow 1} \right) \tilde{\mathbf{S}}_{[j]}^{T_j} \in \mathbb{R}^{d_k \times d_v}. \end{aligned} \quad (17)$$

where $\mathbf{M}_{[i+1]}^{t \leftarrow 1}$ denotes the cumulative transition of the first t local tokens. The first term contains the state generated by the local tokens, whereas the second term propagates the context from preceding ranks through the local KDA updates. At $t = T_{i+1}$, both quantities $\mathbf{M}_{[i+1]}^{T_{i+1} \leftarrow 1}$ and $\tilde{\mathbf{S}}_{[i+1]}^{T_{i+1}}$ can be computed using only the local tokens, before $\mathbf{S}_{[i]}^{T_i}$ is available, and are the fragments each rank exchanges with the others.

The summation in Eq. 17 shows that every state is composed purely from locally computed fragments. These rank-level updates compose associatively, so the incoming state of each rank can be recovered by a prefix scan [78]. Each rank first computes $\mathbf{M}_{[i]}^{T_i \leftarrow 1}$ and $\tilde{\mathbf{S}}_{[i]}^{T_i}$ locally, then exchanges both tensors with one all-gather [140].² After the all-gather, rank $i+1$ reconstructs $\mathbf{S}_{[i]}^{T_i}$ by processing preceding fragments of the same document in order, starting from $\mathbf{S} = \mathbf{0}$ and applying $\mathbf{S} \leftarrow \mathbf{M}_{[j]}^{T_j \leftarrow 1} \mathbf{S} + \tilde{\mathbf{S}}_{[j]}^{T_j}$ at each fragment. Therefore, KCP requires only a fixed-size all-gather for recurrent-state synchronization and achieves linear compute scaling.

5.2 Infra for 3T-class Pre-Training

Kimi K3 pre-training combines Pipeline Parallelism (PP) with virtual stages (VP) [49, 82], Expert Parallelism (EP) [67], ZeRO-1 Data Parallelism [101], Pipeline ZeRO-2 gradient sharding [146], and Context Parallelism (CP, §5.1.2) [51].

²The construction builds on DeltaNet context parallelism [143]. The KDA implementation is available in [FLA PR #691](#).

图：Kimi K3 technical report 第 18 页，说明 KDA Context Parallelism 如何把本地 transition 和 zero-state contribution 组合起来。

KDA-aware prefix cache 的关键是“命中边界必须同时满足 MLA KV 和 KDA state”。K3 把 MLA 的物理 cache block 和 prefix hash block 解耦，例如 physical block 6144 tokens、hash block

512 tokens; KDA checkpoint 只在 hash endpoint 的稀疏子集保存，通常保留会话轮次边界。命中时必须找到同一边界处的 MLA blocks 和所有 KDA cache groups 的 checkpoint。

boundary. We therefore design a KDA-aware prefix cache that manages the two cache types jointly—from a unified paged layout to fine-grained prefix reuse and consistency under concurrent scheduling—keeping million-token prefixes cheap to retain and reusable across requests.

Unified cache layout for hybrid KDA–MLA attention Each Kimi K3 block consists of three KDA layers and one Gated MLA layer, whose caches differ fundamentally. The MLA KV cache grows with sequence length and is paged per token, whereas the KDA recurrent state is fixed in size with a single copy per request. Maintaining a separate manager for each would duplicate the allocation, eviction, and transfer logic. We therefore pack KDA states into the same paged block pool as MLA KV, unifying pages to the same byte size so that both page types share one implementation of allocation, reference counting, and eviction. Within a page, the states of all heads are stored contiguously head by head, so that each head’s byte stream is self-contained and serves as the minimal unit of cross-node transfer. Under prefill/decode disaggregation, when prefill and decode nodes adopt different TP degrees, re-layout is performed on the transfer path with zero GPU-side reshuffling. This asymmetry proved useful during development: any type-confused access yields garbage rather than plausible data — a zero-overhead sanity check on the pooled layout.

KDA prefix cache optimization Block-hash-based prefix caching reuses the KV cache at the granularity of one physical block: only complete blocks are hashed, so only block-aligned prefixes are reusable.

This coupling breaks down in Kimi K3. Block-hash matching requires one block size shared by all layers, and a prefix hit is reusable only if the KDA state at the hit boundary has been persisted. A KDA layer maintains a single large recurrent state per sequence rather than per-token entries, so state snapshots are affordable only at sparse boundaries; the shared block size is therefore forced to 1024–6144 tokens—and, since hashing is tied to the storage block, the hash granularity as well, although MLA’s per-token entries alone would tolerate much finer blocks. At such a coarse granularity caching is nearly useless: requests shorter than one block can never be reused, and chunked prefill exports no cacheable prefix until it crosses a full block boundary.

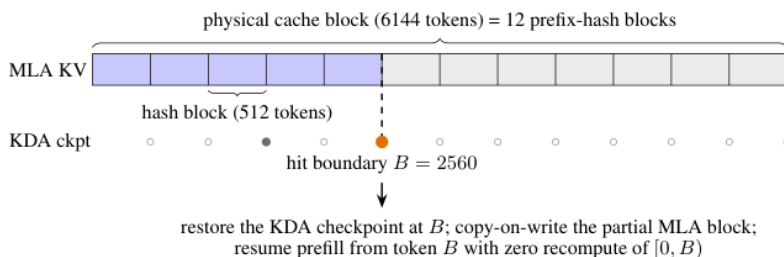


Figure 12: **Fine-grained prefix caching within a physical cache block.** A 6144-token physical block contains twelve 512-token hash blocks, with cached MLA blocks shown in blue and empty blocks in light gray. The markers below show the KDA checkpoint status at each hash boundary. An open circle (○) denotes a boundary without a stored checkpoint, a gray dot (●) denotes a persisted KDA checkpoint, and an orange dot (●) marks the checkpoint hit at $B = 2560$. Persisted checkpoints are sparse and typically coincide with conversation-turn boundaries. The request reuses the five MLA hash blocks and the KDA checkpoint at B , then resumes prefill without recomputing $[0, B)$.

We therefore decouple the two granularities. Prefix hashing runs on fine *hash blocks* (e.g., 512 tokens) inside MLA pages, while the physical block remains the coarse allocation unit. Alignment runs the other way for KDA: checkpoints of the recurrent state are saved only at (a sparse subset of) MLA’s hash endpoints—the only positions a lookup can ever reference.

During prefill, a partially filled MLA page is registered in the prefix-cache index under the chained hash of its last complete hash block, where each hash covers all preceding hash blocks so that matching an endpoint certifies the whole prefix up to it; the registered endpoint advances as the page fills. Meanwhile, after each forward pass, the KDA kernel updates the recurrent state at the last hash-aligned position processed. Checkpoints are large, so intermediate checkpoints superseded as the request advances are recycled, while those at conversation-turn boundaries are retained for cross-request reuse. Cached checkpoints are read-only snapshots: a hit restores the state by copying it into the request’s private running state before the next forward pass, and new checkpoints are written to fresh slots, so a checkpoint visible to other requests is never mutated in place.

Lookup proceeds in two stages (Fig. 12). The MLA stage matches whole physical blocks by chained hash and, at the first missing block, falls back to the hash endpoints inside it, so partially filled pages remain hittable. The KDA stage then requires a checkpoint at the candidate boundary in every KDA cache group, each of which maintains an

图：Kimi K3 technical report 第 23 页，Figure 12 展示 KDA checkpoint 与 MLA hash blocks 的细粒度 prefix cache。

1.6 KDA 的收益和风险

维度	收益	风险或代价
训练	1M context 训练更可行，KCP 让 recurrent state 可以跨设备并行	KDA recurrence 比 attention 更依赖专用 kernel，普通实现容易慢
prefill	fixed recurrent state 降低长上下文 cache 压力	chunkwise kernel、context parallelism、state checkpoint 都需要实现支持
decode	状态大小固定，理论上比长 KV cache 更友好	speculative decoding 回滚复杂，需要 replay projected inputs 或保存额外状态
prefix cache	prefix reuse 可扩展到 hybrid KDA-MLA	只有 MLA 与所有 KDA groups 在同一边界都命中才可复用

对推理服务落地来说，KDA 不能按普通 MHA/MLA 模型处理。接入 K3 时至少要检查：

检查项	为什么重要
vLLM 是否支持 KDA layer 和 KDA state	否则模型即使能加载权重，也可能无法正确推理
prefix cache 是否 KDA-aware	普通 KV block hash 不足以复用 KDA state
P-D 分离时 KDA state 如何传输	prefill/decode TP degree 不同会触发 re-layout
speculative decoding 是否支持状态回滚	draft token 被拒绝后 recurrent state 不能简单倒退

2. Attention Residuals：把 attention 从 token 维度扩展到 layer 维度

2.1 K3 为什么需要 AttnRes

K3 有 93 层、1M context、hybrid KDA-MLA attention。随着深度增加，普通 residual connection 会把历史层信息压缩进当前 hidden state，早期层表示是否能被后层保真读取，取决于中间层变换是否保留这些信息。

K3 report 对这个问题的类比很明确：标准 residual connection 在深度维度上类似 RNN，它把所有之前的信息压成一个状态；Transformer 当年用 attention 替代时间维度 RNN，让每个位置能选择性访问历史位置；AttnRes 则把同样思想搬到深度维度，让每层选择性访问前序层表示。

2.2 Full AttnRes 的定义

对第 l 层，K3 定义一个 layer-specific learnable pseudo-query：

$$q_l = w_l \in \mathbb{R}^d$$

前序层的 keys 和 values 为：

$$k_i = v_i = \begin{cases} h_1, & i = 0 \\ f_i(h_i), & 1 \leq i \leq l-1 \end{cases}$$

其中 h_1 是 token embedding， $f_i(h_i)$ 是第 i 层输出。注意力核为：

$$\phi(q, k) = \exp(q^\top \text{RMSNorm}(k))$$

权重和输出为：

$$\alpha_{i \rightarrow l} = \frac{\phi(q_l, k_i)}{\sum_{j=0}^{l-1} \phi(q_l, k_j)}$$
$$h_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} v_i$$

这个机制的核心不是“把所有残差相加”，而是“让当前层学习应该读哪些历史层”。

RMSNorm 的作用是避免某些大幅值层输出仅因范数大而支配权重。

2.3 Block AttnRes：K3 的可部署版本

Full AttnRes 的算术成本在 $L < 100$ 时还可接受，但真正麻烦是 memory 和 pipeline parallelism 下的跨 stage 通信：需要保留所有前层输出，开销为 $O(Ld)$ 。

K3 使用 Block AttnRes：把 L 层分成 N 个 blocks，每个 block 内把层输出聚合成一个 block representation。对 block n ，若层集合为 B_n ，则：

$$b_n = \sum_{j \in B_n} f_j(h_j)$$

第 n 个 block 的第 i 层看到的 value matrix 为：

$$V = \begin{cases} [b_0, b_1, \dots, b_{n-1}]^\top, & i = 1 \\ [b_0, b_1, \dots, b_{n-1}, b_n^{i-1}]^\top, & i \geq 2 \end{cases}$$

其中 $b_0 = h_1$ ，即 embedding 始终作为一个 source。这样内存/通信开销从：

$$O(Ld) \rightarrow O(Nd)$$

K3 采用 $N \approx 8$ 的经验设置：93 层分成 8 个 blocks，block size 约 12 层，加上 embedding 后共有 9 个 block-level sources。

2.4 AttnRes 与 KDA 的关系

KDA 解决的是 token sequence 维度，AttnRes 解决的是 network depth 维度。两者不是替代关系。

问题	KDA 的回答	AttnRes 的回答
历史 token 怎么保留	用 recurrent state 压缩长程上下文	不直接解决 token 维度
历史层表示怎么保留	不直接解决层间压缩	用 pseudo-query 选择性读前序层/block 表示
为什么 K3 同时需要两者	1M context 需要序列维度高效	93 层深模型需要深度维度信息流

更具体地说，KDA 把百万 token 的历史压入固定状态，会不可避免地损失部分可精确检索的信息；周期性 Gated MLA 提供全局 token-token 交互，而 AttnRes 让模型在深度方向上重新访问较早层的表示。三者一起服务于“长序列 + 深网络”的信息保真。

2.5 AttnRes 的 serving kernel

K3 report 说明 Block AttnRes 在 serving 中采用两阶段：

阶段	做法
inter-block pass	批量读取 cached block representations，每个 block 读一次
intra-block merge	通过 online softmax 合并当前 block 的 partial sum

Prefill 时，为避免每个 TP rank 都 materialize block representations，K3 使用 sequence parallelism：把 TP all-reduce 拆成 reduce-scatter 和 all-gather，并把 intra-block kernel 插在二者之间，让每个 token 的 block representation 只在一个 rank 上物化。

Decode 时，inter-block kernel 放到 side stream，与 main stream 的独立计算重叠；intra-block merge、partial-sum update 和后续 RMSNorm 融进前面的 TP all-reduce，减少单独 kernel 和内存读写。

independent recurrent state. The hit is the longest boundary satisfying both stages—always a multiple of the hash block, and never required to be a multiple of the physical block. In Fig. 12, a request whose first 2800 tokens match the cached prefix hits at $B = 2560 = 5 \times 512$, deep inside a 6144-token physical block, and resumes prefill from token B instead of recomputing $[0, B)$.

Consistency under concurrent scheduling The remaining design points are each dictated by a concrete failure mode of sharing partially filled blocks, in a setting where a hit block is at once a shared cache entry and the growth point of a private request, and where the MLA and KDA cache groups must agree on every hit boundary. First, all cache groups draw blocks from one shared free list, so allocating a private copy for one group could evict a block that another group has just hit; every hit block is therefore pinned across all groups before anything is allocated. Second, the copy into the private block executes on the GPU immediately before the forward pass, so a block allocated or registered within the current scheduling step would still hand the previous owner's bytes to a reader; such blocks are excluded from matching until their copies land. Third, a checkpoint can restore a request only if it exists in every KDA group, so evicting one group's checkpoint atomically invalidates its siblings — a checkpoint is either hittable in every group or in none. With these mechanisms, every registered state always corresponds to exactly its declared token prefix, and prefix caching for hybrid KDA–MLA models reaches the same generality as for full-attention models: any shared prefix is reusable at any 512-token boundary, independently of request length, chunking, or scheduling interleaving.

5.4.2 High-Performance Kernels

Kimi K3 introduces several new architectural modules: KDA (§2.1.1), Block AttnRes (§2.2), and Stable LatentMoE (§2.3). We optimize the kernel implementation for each.

KDA Compared with KDA prefill (§5.1), KDA decoding presents a distinct set of challenges: the primary bottleneck shifts from exploiting parallelism to efficiently managing the evolving recurrent state, which is updated in place at every decoding step. This in-place update becomes problematic in MTP-based speculative decoding: if verification rejects a subset of the drafted tokens, the state has already advanced beyond the last accepted token and cannot be trivially rolled back. Maintaining a state snapshot for each draft position would enable rollback, but would also multiply state traffic — a cost that dominates at the large batch sizes typical of online serving.

The state after any accepted draft prefix, however, is fully determined by the projected inputs of the draft tokens, which are far smaller than the state itself. We therefore cache only these projected inputs, rebuild the states of accepted tokens on-chip, and write back the states of the verified and bonus tokens, a design independently proposed in the concurrent work ReplaySSM [25]. The replayed tokens, the bonus token, and the next draft window share one recurrent loop inside a single fused kernel covering short convolution, input normalization, gating, the KDA recurrence, and output normalization. Verification latency grows sub-linearly with the number of tokens verified and remains below that of state-caching baselines. Because the projection caches never leave the decode stage, prefix caching and prefill–decode disaggregation operate on the same payload as in non-speculative serving.

Block AttnRes Block AttnRes [58] follows a two-phase schedule: a batched inter-block pass reads the cached block representations once per block, after which each layer folds in the intra-block partial sum through an online-softmax merge [80]. Memory access accounts for a substantial fraction of the cost of these kernels in both prefill and decoding, so our optimizations in both stages focus primarily on memory efficiency.

For prefill, materializing the block representations on every tensor-parallel (TP) rank would incur substantial redundant memory consumption. We therefore adopt sequence parallelism (SP) for activations: the TP all-reduce is decomposed into a reduce-scatter and an all-gather, with the intra-block kernel inserted between the two collectives, operating on the sequence-sharded hidden states so that the block representations of each token are materialized on exactly one rank. This eliminates the additional memory consumption and reduces the I/O overheads of Block AttnRes during prefill.

For decoding, we launch the inter-block kernel on a side stream so that it overlaps with independent computation on the main stream. The intra-block kernel is instead streamlined through fusion: the merging of the AttnRes output with its partial-sum update, together with the subsequent RMSNorm, is fused into the preceding TP all-reduce, eliminating a dedicated kernel for the intra-block phase. Together, these optimizations hide the latency of the inter-block pass and reduce the memory traffic of the intra-block phase.

Stable LatentMoE Stable LatentMoE increases both the total number of experts and the number of activated experts per token. The resulting growth in both the expert space and the per-token expert count raises scheduling and coordination overheads, making it difficult for conventional MoE kernels to sustain high hardware utilization. These challenges motivate dedicated kernel optimizations for this module.

图：Kimi K3 technical report 第 24 页，包含 KDA、Block AttnRes、Stable LatentMoE 的 serving kernel 优化入口。

2.6 AttnRes 的收益和边界

维度	收益	边界
深层信息流	避免所有历史层信息被普通 residual 统一压缩	公开报告没有给 AttnRes 单独 ablation
长程任务	早期层捕获的局部证据、格式状态、工具状态更容易被后续层读取	对 coding、BrowseComp、long-context reasoning 的单项贡献未公开
系统实现	Block AttnRes 把开销控制到 $O(Nd)$ ，并有专门 serving kernel	vLLM 或其他推理框架是否支持完整 Block AttnRes 仍需确认

K3 report 的 case study 提到，Kimi K3 在 GPU kernel optimization 任务中把 AttnRes latency 从 283.6 ms 降到 114.4 ms。但这属于 K3 作为 coding agent 优化 kernel 的能力案例，不等价于 AttnRes 模块对模型质量的消融证据。

3. Stable MoE：让 896 experts / Top-16 的宽度扩展稳定下来

3.1 K3 的 MoE 扩展为什么需要“stable”

K3 的专家配置比 K2 大很多：

项	Kimi K2	Kimi K3
routed experts	384	896
active experts per token	8	16
shared experts	1	2
MoE hidden dimension per expert	2048	3072
Latent MoE dimension	无	3584
sparsity	$384/8 = 48$	$896/16 = 56$

专家池扩大带来两个直接问题：

问题	具体表现
计算链更不稳定	routed path 经过 down projection、gated expert FFN、up projection，近似四个矩阵乘连续组合，2.8T 规模下内部 activation 容易爆炸
负载均衡更难	896 experts 的 router bias 更新和专家负载波动进入大规模分布式训练瓶颈，专家过热或死亡都会拖慢 EP 训练

所以 K3 的 Stable MoE 不是一个单独小技巧，而是一组组合设计：LatentMoE 负责把 routed path 变窄，RMSNORM 和 SiTU-GLU 负责抑制激活异常，Quantile Balancing 负责路由负载均衡，MoonEP 和 serving kernels 负责系统侧负载与通信。

3.2 Stable LatentMoE 的结构

LatentMoE 的基本思想是把 full model width 和 routed expert width 分离：

路径	作用
shared experts	处理 full-width token representation，保留通用变换能力
routed experts	只在 latent width ℓ 上工作，降低专家扩展时的通信和 expert-weight traffic

K3 report 给出的 Stable LatentMoE 形式是：

$$u = \sum_{i \in T_k(x)} p_i E_i^{\text{routed}}(W_{\downarrow} x)$$

$$y = \sum_{j=1}^{N_s} E_j^{\text{shared}}(x) + W_{\uparrow} \text{RMSNorm}(u)$$

其中：

符号	含义
$W_{\downarrow} x \in \mathbb{R}^{\ell}$	把 full-width 输入投到 routed latent space
$E_i^{\text{routed}} : \mathbb{R}^{\ell} \rightarrow \mathbb{R}^{\ell}$	routed expert，在 latent width 上工作
$E_j^{\text{shared}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$	shared expert，保留 full-width path
p_i	router weight，由 Quantile Balancing 路由规则定义
$N_s = 2$	K3 每层固定两个 shared experts

RMSNorm 插在 routed aggregation 和 up-projection 之间，目的不是为了好看，而是降低 u 的尺度波动对后续 $W_{\uparrow}$ 和 shared branch 合流的影响。K3 report 明确说这既稳定训练，也持续改善 validation loss 和 downstream benchmarks。

3.3 SiTU-GLU：把 SwiGLU 的大值爆炸封顶

SwiGLU 的经验效果强，但两个 multiplicative factors 都是无界的。大模型和低精度训练中，如果 gate branch 和 up branch 同时出现大坐标，乘积容易制造 activation outliers。

K3 提出 Sigmoid Tanh Unit GLU (SiTU-GLU)：

$$\text{SiTU-GLU}(x) = \left[\beta_1 \tanh \left(\frac{W_g x}{\beta_1} \right) \odot \text{Sigmoid}(W_g x) \right] \odot \left[\beta_2 \tanh \left(\frac{W_u x}{\beta_2} \right) \right]$$

K3 使用：

$$\beta_1 = 4, \quad \beta_2 = 25$$

因此每个输出坐标满足：

$$\|\text{SiTU-GLU}(x)\|_\infty \leq \beta_1 \beta_2 = 100$$

这个设计保留了 SwiGLU 的局部形状：当 z 接近 0 时，

$$\beta \tanh\left(\frac{z}{\beta}\right) = z + O\left(\frac{z^3}{\beta^2}\right)$$

所以 SiTU-GLU 在原点附近与 SwiGLU 一阶一致；当输入很大时，tanh smooth cap 把输出封顶。相比 hard clamp，smooth cap 在饱和边界外仍有非零梯度，训练行为更平滑。

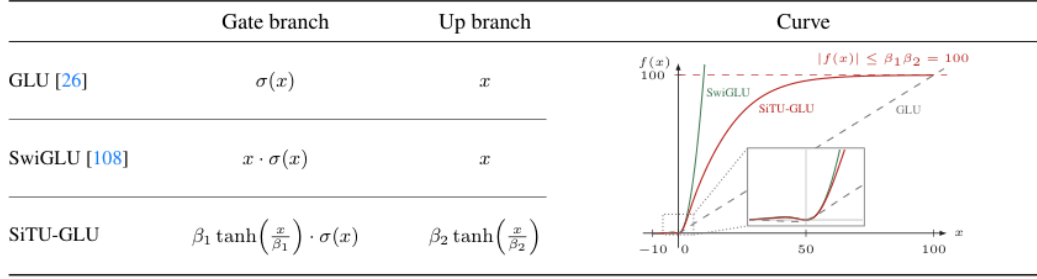


Figure 4: Gate and up branches of GLU, SwiGLU, and SiTU-GLU, together with their scalar responses, where σ denotes the sigmoid function. Both branches receive the scalar input x , and all curves share the domain $x \in [-10, 100]$; the inset magnifies the near-origin region. SiTU-GLU, shown in red with $\beta_1 = 4$ and $\beta_2 = 25$, closely follows SwiGLU near the origin and approaches the bound $|f(x)| \leq \beta_1 \beta_2 = 100$ for large positive inputs, whereas SwiGLU remains unbounded.

bias updates remain well behaved. Stable LatentMoE addresses these two failure modes with three components: an RMSNorm before the up-projection and Sigmoid Tanh Unit GLU (SiTU-GLU) to suppress activation explosion, and Quantile Balancing (QB) for load balancing.

As illustrated in Fig. 2, the layer follows the shared- and routed-expert organization of DeepSeekMoE [23]. For $x \in \mathbb{R}^d$, the shared experts process x directly, while the routed path projects it to $z = \mathbf{W}^\downarrow x \in \mathbb{R}^\ell$, dispatches z to the selected experts, and maps their weighted aggregate back to $\mathbb{R}^d$ through $\mathbf{W}^\uparrow$:

$$\begin{aligned}
 \mathbf{u} &= \sum_{i \in \mathcal{T}_k(x)} p_i E_i^{\text{routed}}(\mathbf{W}^\downarrow x), \\
 \mathbf{y} &= \sum_{j=1}^{N_s} E_j^{\text{shared}}(x) + \mathbf{W}^\uparrow \text{RMSNorm}(\mathbf{u}).
 \end{aligned} \tag{11}$$

Here, $\mathbf{u} \in \mathbb{R}^\ell$ is the aggregated routed representation, $E_j^{\text{shared}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $E_i^{\text{routed}} : \mathbb{R}^\ell \rightarrow \mathbb{R}^\ell$ are the shared and routed expert feed-forward networks, and p_i is the router weight defined by the Quantile Balancing rule below. Kimi K3 fixes the number of full-width shared experts to $N_s = 2$ in every layer.

2.3.1 Normalized LatentMoE

The original LatentMoE directly applies $\mathbf{W}^\uparrow$ to the aggregated routed representation $\mathbf{u}$, whose scale can vary with the selected experts and their routing weights. As shown in Eq. 11, Kimi K3 instead inserts RMSNorm [147] between expert aggregation and the up-projection. This normalization reduces the sensitivity of the routed branch to scale variation before it is combined with the full-width shared branch. Beyond stabilizing training, the additional RMSNorm consistently improves validation loss and downstream benchmarks.

2.3.2 Sigmoid Tanh Unit GLU

Gated Linear Units (GLUs) modulate a linear value branch with a sigmoid-activated gate, computing $\text{Sigmoid}(\mathbf{W}_g x) \odot \mathbf{W}_u x$ [26]. SwiGLU replaces the sigmoid gate with $\text{Swish}(x) = x \text{Sigmoid}(x)$ and yields strong empirical performance in Transformers [108]. SwiGLU has subsequently become a widely adopted FFN design in large language models, while a complete account of its empirical effectiveness remains open.

However, both multiplicative factors in SwiGLU are unbounded, so coincident large coordinates can produce activation outliers and increase overflow risk in low-precision arithmetic. The sigmoid gate of the original GLU avoids unbounded gate growth, but it does not retain the approximately linear positive regime of Swish. This motivates an activation that controls large-value growth while preserving the characteristic local and positive-side response of SwiGLU. Other recent efforts have explored alternative parameterizations of this trade-off [52].

To satisfy these requirements, we propose Sigmoid Tanh Unit GLU (SiTU-GLU). SiTU-GLU applies the smooth cap $\text{softcap}(x, \beta) = \beta \tanh(x/\beta)$ to the linear factor of the Swish gate and independently to the up branch:

$$\text{SiTU-GLU}(x) = \left[\beta_1 \tanh\left(\frac{\mathbf{W}_g x}{\beta_1}\right) \odot \text{Sigmoid}(\mathbf{W}_g x) \right] \odot \left[\beta_2 \tanh\left(\frac{\mathbf{W}_u x}{\beta_2}\right) \right], \tag{12}$$

图：Kimi K3 technical report 第 7 页，Figure 4 展示 GLU、SwiGLU、SiTU-GLU 的 branch 定义和曲线，并给出 Stable LatentMoE 的公式。

3.4 Quantile Balancing：不用 auxiliary loss 的专家负载均衡

传统 MoE 常用 auxiliary load balancing loss。问题是 loss 权重需要调，且可能和主任务优化相互拉扯。K3 采用 auxiliary-loss-free routing：给每个专家一个 bias b_j ，用它影响 Top-k dispatch，但不把 bias 放进 mixture weight。

对 token x_i ，router score 为：

$$s_i = \text{Sigmoid}(W_r x_i)$$

K3 的 routing rule 是：

$$T_i = \text{argtopk}(s_i + b)$$

$$p_{i,j} = \frac{s_{i,j}}{\sum_{r \in T_i} s_{i,r}}, \quad j \in T_i$$

注意 b 只影响被派发到哪些专家，不影响 $p_{i,j}$ 的归一化权重。这样 dispatch 可以均衡，但 router 的梯度目标不被 bias 直接扭曲。

设 batch 中有 m 个 tokens、 n 个 routed experts、Top-k，则每个专家的目标负载为：

$$q = \frac{mk}{n}$$

Quantile Balancing 的思路是：先做 Top- $(k + 1)$ ，得到每个 token 的 cutoff $\alpha_i^{(t)}$ ，再看每个专家 j 的 margin：

$$s_{:,j} - \alpha^{(t)}$$

下一步 bias 由分位数确定：

$$\hat{b}_j^{(t+1)} \leftarrow -\text{quantile}_{1-k/n} \left(s_{:,j} - \alpha^{(t)} \right)$$

$$b^{(t+1)} \leftarrow \hat{b}^{(t+1)} - \text{mean}(\hat{b}^{(t+1)}) \mathbf{1}$$

直觉上，QB 为每个专家选择一个阈值，让它刚好接收目标数量 q 的 token。K3 report 的小例子中， $m = 8, n = 4, k = 1$ ，普通 Top-k 得到负载 $(4, 3, 1, 0)$ ，QB 后变成 $(2, 2, 2, 2)$ 。

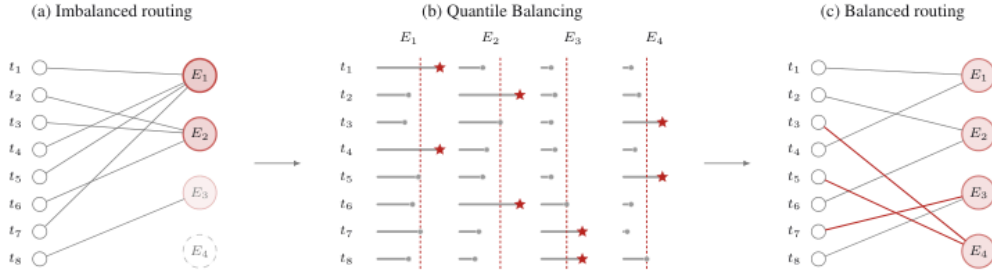


Figure 5: Illustration of Quantile Balancing with $m = 8$ tokens, $n = 4$ routed experts, and $k = 1$ selected expert per token. (a) Token-wise Top- k routing (tokens on the left, experts on the right) produces loads $(4, 3, 1, 0)$; darker circles indicate overheated experts, whereas faded and dashed circles indicate underutilized and dying experts, respectively. (b) Each gray bar is the margin of the currently biased score, $s_{i,j} + b_j^{(t)} - \alpha_i^{(t)}$, so the row-wise maxima reproduce the routing in (a). The dashed red line in each column is the bias adjustment $\hat{b}_j^{(t)} - \hat{b}_j^{(t+1)}$, placed at the $(q+1)$ -th largest margin so that exactly $q = 2$ margins exceed it. The marker $\star$ denotes the row-wise Top- k choice after subtracting the column adjustments, i.e., the routing in (c). (c) The retained choices yield the balanced load $(2, 2, 2, 2)$; red edges denote assignments changed by QB.

For Kimi K3, we set the soft-cap hyperparameters to $\beta_1 = 4$ for the gate branch and $\beta_2 = 25$ for the up branch. The scaled tanh is approximately linear near the origin and bounded at large magnitude, allowing SiTU-GLU to preserve the local response of SwiGLU while controlling both factors in the product. Fig. 4 compares the branch definitions and scalar responses of GLU, SwiGLU, and SiTU-GLU on a common slice.

§ B gives the local expansion, limiting case, formal output bound, and comparison with hard clamping.

2.3.3 Quantile Balancing

Unlike auxiliary-loss-based routing [33], Kimi K3 adopts auxiliary-loss-free routing [30]. Load balancing is implemented by adding an expert-specific bias b_j to the router score used for Top- k selection. For token x_i , the router computes $s_i = \text{Sigmoid}(\mathbf{W}_r x_i)$ and applies

$$\mathcal{T}_i = \text{argtop}_k(s_i + \mathbf{b}), \quad p_{i,j} = \frac{s_{i,j}}{\sum_{r \in \mathcal{T}_i} s_{i,r}}, \quad j \in \mathcal{T}_i. \quad (13)$$

Because $\mathbf{b}$ is omitted from $p_{i,j}$, it regulates dispatch without altering the mixture weights or the gradient-based optimization of the router. The original method updates $\mathbf{b}$ with the fixed-step rule $\hat{b}_j^{(t+1)} = \hat{b}_j^{(t)} + \gamma \text{sign}(\bar{\ell} - \ell_j^{(t)})$ [30], for which γ trades off slow adaptation against load oscillation. Maintaining balanced loads becomes more challenging as LatentMoE increases the routed expert pool to 896 per layer. Imbalanced routing slows expert-parallel training and may leave some experts poorly trained [48].

To address this limitation, we introduce Quantile Balancing (QB), which sets each expert bias from the router-score quantile that matches its target load [112]. Consider a training batch of m tokens routed to n experts with Top- k selection, so the target load is $q := mk/n$ tokens per expert. QB derives the next bias from a single forward pass. Routing replaces the Top- k selection with Top- $(k+1)$ on the biased score $s_i + \mathbf{b}^{(t)}$: the first k entries are the routes actually taken, while the $(k+1)$ -th entry is the cutoff $\alpha_i^{(t)}$ that an expert must exceed to enter token i 's Top- k . Taking the cutoff from Top- $(k+1)$ routing avoids a separate token-side quantile. We then choose each expert bias so that expert j receives its target load: with the cutoffs fixed, the token count routed to expert j under a candidate bias $\hat{b}_j^{(t+1)}$ is

$$\sum_{i=1}^m \mathbf{1}[s_{i,j} + \hat{b}_j^{(t+1)} > \alpha_i^{(t)}],$$

which is monotonically decreasing in the threshold $-\hat{b}_j^{(t+1)}$. Assuming no ties, setting this count to q makes $-\hat{b}_j^{(t+1)}$ the $(q+1)$ -th largest margin $s_{i,j} - \alpha_i^{(t)}$, so that exactly q margins stay above the threshold. Since $q/m = k/n$, this is

图：Kimi K3 technical report 第 8 页，Figure 5 展示 Quantile Balancing 如何把不均衡路由变成均衡负载。

3.5 大规模训练中的 QB histogram estimator

真实训练中不可能把全局 batch 中所有 margin gather 到一起做精确分位数。K3 使用 histogram estimator：

步骤	做法
本地统计	每个 rank 为每个 expert 统计 margin histogram
全局汇总	用一次 all-reduce 求和各 rank bin counts
近似分位数	从 pooled counts 恢复全局 batch 的 quantile
通信成本	每个 expert 只传几百个 bins，而不是传所有 token-expert margins

这个设计的关键是 counts 可加，所以不受 token shard 方式影响；代价是存在 bin width 带来的估计误差。

3.6 Stable MoE 的系统侧：MoonEP 与 serving kernel

Stable MoE 在模型侧让路由更稳，在系统侧还需要让 EP 执行更可预测。

K3 report 对 MoonEP 的描述是：每个 rank 接收固定 $S \times K$ tokens，并在每个 rank 最多 E/R 个 redundant experts 的条件下保持 perfect balance。系统实现包括：

机制	作用
online planning kernel	近似求解专家分配，避免 CPU/host 同步成为瓶颈
zero-copy communication	直接把 tokens 发到 remote rank 的 expert-grouped positions
static shapes	消除 per-layer MoE host synchronization
bounded redundant experts	用有限冗余换取固定负载和更稳定流水

Serving 侧，K3 对 Stable LatentMoE 还做了专门 kernel：

优化	作用
fuse latent down-projection with router	把 latent $W_{\downarrow}$ 和 router 合并成一个 GEMM
shard latent weight matrices	降低 latent GEMM 的冗余权重流量

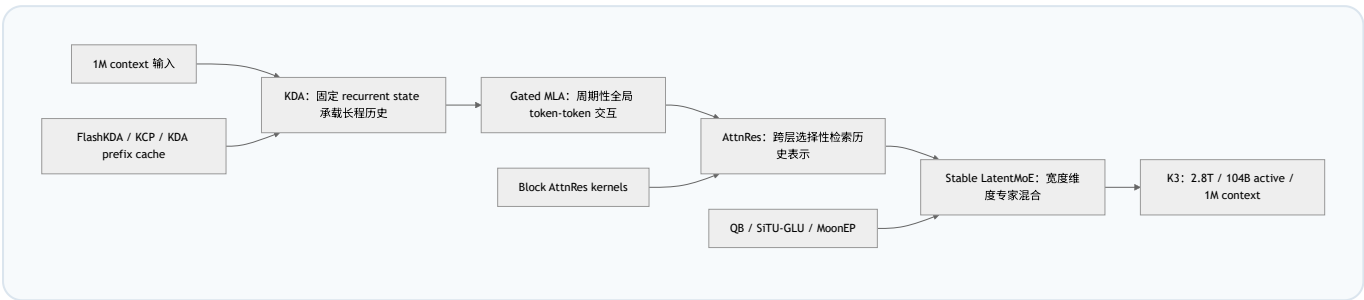
优化	作用
fuse output all-gather into GEMM epilogue	用 multimem store 指令减少额外通信开销
overlap communication with shared experts	把 latent path 的通信藏在 shared-expert computation 后面
token-centric MoE decoding kernel	小 batch decode 下 group GEMM 变成 weight streaming，使用 WarpDecode 风格降低内存瓶颈

3.7 Stable MoE 的收益和风险

维度	收益	风险或边界
模型容量	896 experts / Top-16 提供更大的专家特化空间	单独归因到 expert 数、Top-k、latent width 的 ablation 未完全公开
训练稳定性	RMSNorm、SiTU-GLU、QB 共同控制 scale、activation、load	SiTU-GLU 的 β_1, β_2 是否跨模型可迁移需要验证
分布式效率	QB + MoonEP 让 EP 负载更可预测	真实 expert load histogram、rank dispatch 方差和 tail latency 需要实测
服务实现	fused kernels 和 token-centric decode 降低 MoE serving 开销	vLLM 是否完整支持 K3 Stable LatentMoE、MoonEP、MXFP 格式仍需跟进

4. 三项技术的组合关系

K3 的三项技术不是孤立模块，而是按 token、layer、channel 三个混合维度协同。



组合	关键含义
KDA + Gated MLA	KDA 提供低 cache 成本的长程状态，Gated MLA 周期性保留全局精确交互
KDA + AttnRes	一个解决序列维度，一个解决深度维度，避免 1M token 和 93 层同时带来的信息瓶颈
Stable MoE + AttnRes	MoE 扩宽能力空间，AttnRes 让深层模块不必只依赖局部 residual 累积
Stable MoE + MoonEP	模型侧路由均衡和系统侧专家执行均衡必须配套，否则 896 experts 会转化为 tail latency

一句话概括：K3 的核心不是“用了 KDA”或“专家更多”，而是把长序列、深层网络、超大家池三个方向同时扩展，并为每个方向配了相应的训练和 serving 系统。

5. 工程落地与推理实验关注点

5.1 接入 K3 前必须确认的实现问题

模块	检查项	不确认的风险
KDA	模型 config、KDA kernel、KDA state layout、KDA prefix cache	普通 KV cache 逻辑无法复用 KDA state，长上下文会退化或错误
Gated MLA	NoPE MLA、full-rank gate、FP32 attention output training 逻辑是否影响推理实现	以 K2 MLA 假设直接加载可能漏掉门控路径
AttnRes	Block AttnRes state、block source 数、kernel fusion	标准 residual path 无法等价替代 AttnRes
Stable LatentMoE	latent routed width、shared/routed experts、QB frozen bias、SiTU-GLU、量化格式	普通 MoE expert mapping 和 activation function 可能不兼容
EP / serving	expert parallel、rank dispatch、KDA/MLA cache transfer、P-D 分离 re-layout	平均吞吐看起来可用，但 p99 latency 和 cache miss 成本不可控

5.2 推荐 benchmark 指标

方向	指标
KDA 长上下文	TTFT、prefill tokens/s、decode TPOT、KDA state transfer bytes、prefix cache hit boundary、cache miss recompute tokens
AttnRes	block 数、block state bytes、prefill/decode extra latency、kernel launch 数、TP all-reduce 融合效果
Stable MoE	expert load variance、rank dispatch bytes、热门专家 p99、token-centric decode kernel 吞吐、small-batch latency
三者组合	1M context 下 quality/cost ratio、长程 coding 成功率、BrowseComp/DeepSearch 类型任务 wall time、cache-aware routing 收益

5.3 建议实验

实验	做法	目的
KDA prefix cache 微基准	构造共享前缀长度为 512、2560、6144、128K 的请求，对比 cache hit/miss	验证 KDA checkpoint 与 MLA hash block 边界对 TTFT 的影响
MoE tail latency 压测	构造不同 batch token 分布，记录 expert load 和 p99 decode latency	验证 Stable MoE 是否仍会出现专家拥塞
AttnRes kernel 开销	分别测打开/关闭或模拟 AttnRes block state 的额外内存与 latency	判断 Block AttnRes 对 serving 的实际成本
1M coding workload	使用长 repo prompt + 多轮工具调用，记录 cache hit、工具步、最终测试通过率	观察 KDA、AttnRes、Stable MoE 对长程 coding 的组合收益

6. 参考链接

来源	链接
Kimi K3 官方博客	https://www.kimi.com/blog/kimi-k3
Kimi K3 Hugging Face	https://huggingface.co/moonshotai/Kimi-K3
Kimi K3 GitHub	https://github.com/MoonshotAI/Kimi-K3
Kimi K3 technical report PDF	https://github.com/MoonshotAI/Kimi-K3/blob/main/k3_tech_report.pdf
InfraTech Kimi K3 architecture	https://github.com/CalvinXKY/InfraTech/blob/main/models/kimi_k_3/kimi_k_3_architecture
Kimi Linear GitHub	https://github.com/MoonshotAI/Kimi-Linear
Kimi Linear paper	https://huggingface.co/papers/2510.26692
FlashKDA	https://github.com/MoonshotAI/FlashKDA

来源	链接
MoonEP	https://github.com/MoonshotAI/MoonEP
Gated Delta Networks	https://arxiv.org/abs/2412.06464
LatentMoE	https://arxiv.org/abs/2601.18089
BASE Layers	https://arxiv.org/abs/2103.16716
DeepSeek-V2 MLA	https://arxiv.org/abs/2405.04434
Transformer	https://arxiv.org/abs/1706.03762
ResNet	https://arxiv.org/abs/1512.03385